CHECKPOINT 3

Data Transformation Notebook and Agile Approach

Team 11 – Road Accident Risk Pipeline

Course	Data Engineering and Big Data Analytics, PGDM Business Analytics
Client brief	Accenture, Data Engineer (Hyderabad)
Submitted	7 November 2026
Git repository	https://github.com/prashansakpn/T11-us-accidents-pipeline
Team members	Prashansa Naomi Konduru (BA05042), Sandeep Thummalapalli (BA05052), Lakkineni Roshith (BA05032), Leira George (BA05062), Chrislynn Sabu (BA05017), Robin Miranda (BA05048)

1. What We Built

The repository holds our notebook, saved with the outputs of our full run, and a README that explains how to run it again. The notebook works on the US Accidents dataset (7,728,394 records, 46 columns; Moosavi et al., 2019; licence CC BY-NC-SA 4.0) in Databricks Free Edition. It first loads the raw file, untouched, into a **bronze** table, and then cleans that data into a **silver** table that is ready for analysis.

Step	What happened on the full run
Bronze	All 7,728,394 rows loaded exactly as they came, with a record of when and from which file; the count matches the published figure.
Silver	Every column given its proper type, the two date formats read correctly, year, month, weekday, hour and accident duration added, and over 70 weather descriptions grouped into 8 categories.
Checks	No duplicate IDs, and no row broke our four rules (unreadable date, severity outside 1–4, end before start, location outside the US).
Missing values	Precipitation is blank in 28.51% of rows and wind speed in 7.39%; visibility, humidity, weather and temperature are blank in about 2% each.

We were a little surprised that no rows were rejected. It seems the dataset's authors had already kept severity, dates and locations clean, so our checks confirmed this rather than finding problems. The real issue in this data is missing values, especially precipitation. We left those blanks as they are, because filling them with guesses would make our later findings look more certain than they really are.

2. Why We Chose an Agile Approach

When we started, we did not know what problems the data would throw at us; we only found out once we loaded it. On top of that, Databricks Free Edition stops compute for the rest of the day if a workspace uses too much. Designing everything first and building it all at the end would have been risky. So we work in small steps: build a piece, test it, learn from it, and only then move on.

3. How This Works for Us in Practice

Each checkpoint is a sprint. Every checkpoint ends with something that works and that our instructor reviews, and we use the feedback to plan the next one.

Sprint	What we deliver	Skill gaps it works on	Status
1 (CP1)	Our chosen job posting and the skill-gap matrix rated by all six of us	Shows us what to learn	Done
2 (CP2)	One-page plan: business problem, dataset and licence, pipeline steps	Planning around our gaps	Done
3 (CP3)	Bronze and silver tables and a quality log, on all 7.7 million rows	Spark, lakehouse layers, ETL, data quality	Done
4 (CP4)	Gold tables, dashboard, README and governance note	Spark, lakehouse layers, monitoring	Next
Final	Report, recommendation and presentation	Communication	Planned

Our roles. Each of us holds a role from the project guide. Prashansa, as Project Captain, keeps us on track and speaks for our client, the insurer's pricing team. The rest of us build and check the pipeline as Data Wranglers, Analyst, Governance Guardian and Storyteller. After this checkpoint we swap roles, so everyone gets to try something new.

Tasks written from the client's side. We describe our work as short user stories, which reminds us why each task matters. For example:

- "As a pricing analyst, I want every accident to have a clean timestamp, so that I can see the peak hours." (done this sprint)
- "As a pricing analyst, I want the weather grouped into a few clear types, so that I can compare how severe accidents are in each." (done this sprint)
- "As a pricing manager, I want the share of severe accidents by state and city, so that I can price risk by region." (next sprint)

When a step counts as done. We agreed that a step is finished only when it runs on the full data, the row counts add up, every quality check passes or has an explanation, everyone in the team can explain the code, and it is saved in GitHub.

Meeting and learning. We have agreed to hold a short weekly stand-up where each of us says what we did, what is blocking us and what comes next. Our shared learning log is where we write down what broke and how we fixed it, so it works as an ongoing retrospective.

Starting small. We built and tested every step on a 1% sample (77,145 rows) before running it on all 7.7 million rows. This kept our compute use low and meant mistakes were quick and cheap to fix.

4. Looking Back at This Sprint

Area	What happened
What went well	The full run worked: all 7,728,394 rows went through bronze and silver, and every run now saves its quality results automatically.
First problem	Uploading the 3 GB file through the browser failed. We uploaded the much smaller zip file (685 MB) instead and unzipped it inside Databricks.
Second problem	In our 1% test the row counts did not add up: we seemed to end with more rows than we started with. Databricks was taking a new random sample every time we counted, so each step saw slightly different rows. Saving the sample once as a table fixed it. Our own row-count check caught this, which showed us that building checks in from the start is worth the effort.
What we will change	Next sprint we will add a row-count check to every new table from day one, keep testing on samples first, and save our work to GitHub after every step that works.

Reference: Moosavi, S., Samavatian, M. H., Parthasarathy, S., & Ramnath, R. (2019). A Countrywide Traffic Accident Dataset. arXiv:1906.05409.